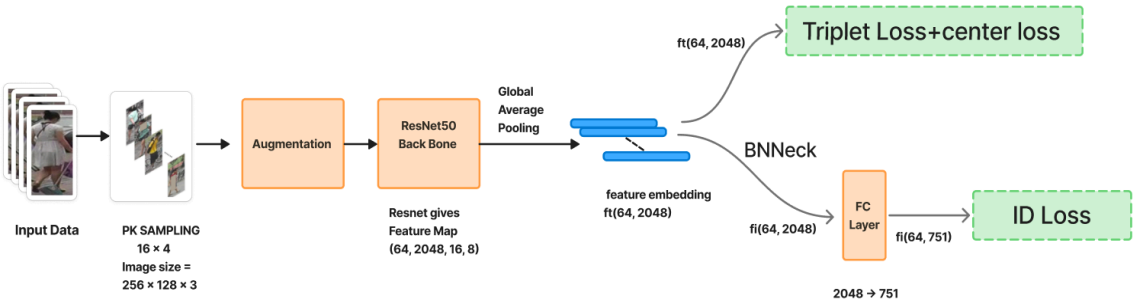
Overview of the Architecture I Used in the ResNet Pipeline :



Train Pipeline

I used a ResNet50 backbone pretrained on ImageNet. Input images are sampled using PK sampling (P = 16 identities × K = 4 images = batch size of 64) and passed through standard augmentations including random flip, random crop, and random erasing.

The backbone outputs a spatial feature map which is compressed via Global Average Pooling (GAP) into a 2048-dimensional vector called *ft*. This raw feature *ft* is directly used for Triplet loss and Center loss during training.

The feature *ft* then passes through BNNeck (a single BatchNorm layer) to produce the normalized feature *fi*. The normalized feature *fi* is used for the ID loss (cross-entropy with label smoothing) during training.

Based on insights from the literature, I incorporated several training strategies to strengthen the baseline model. I used a warmup learning rate at the beginning of training to gradually increase the learning rate, which helps stabilize early optimization and improve convergence [3]. I also applied Random Erasing as a data augmentation technique to improve robustness against occlusion and partial visibility in pedestrian images [1]. In addition, I used Label Smoothing in the ID classification loss to reduce model overconfidence and improve generalization [2].

I modified the last stride of the ResNet50 backbone to 1 in order to increase the spatial resolution of the final feature map, which improves the discriminative capability of the learned features. The model also uses the BNNeck structure, where a Batch Normalization layer separates the embedding used for metric learning and classification [3]. Finally, I incorporated Center Loss alongside Triplet Loss to encourage intra-class compactness by pulling embeddings of the same identity closer together in the feature space during training [4].

Inference Pipeline

During inference, the query and gallery images are passed through the trained backbone network to extract their 2048-dimensional embeddings. Cosine similarity is first computed between the query embedding and all gallery embeddings to obtain an initial ranking. This ranking is then refined using k-reciprocal re-ranking, which leverages neighborhood relationships between samples to improve retrieval accuracy.

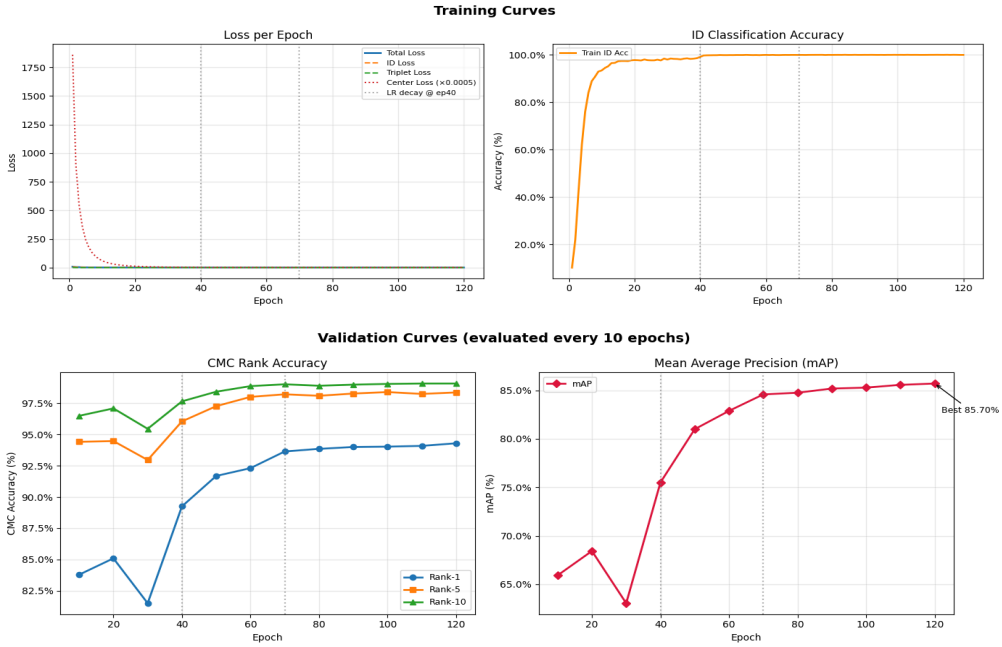
Results

Training Results

The model was trained for 120 epochs using the ResNet50-based ReID pipeline. During training, the model showed consistent improvement in retrieval performance as training progressed. At the end of training (epoch 120), the model achieved **mAP = 85.70%,** indicating stable convergence of the training process.

The final evaluation results at epoch 120 were:

- **Rank-1:** 94.30%
- **Rank-5:** 98.37%
- **Rank-10:** 99.08%



Final Inference Results (Market-1501)

The trained model was evaluated on the Market-1501 dataset during inference. After extracting embeddings for the query and gallery images and applying k-reciprocal re-ranking, the model achieved the following performance:

- **mAP**: 94.08%
- **Rank-1**: 95.46%
- **Rank-5**: 97.80%
- **Rank-10**: 98.34%
- Feature extraction time: 43.20 s
- Re-ranking time: 51.71 s
- Total evaluation time: 94.90 s
- Peak GPU memory usage: 1.815 GB

These results demonstrate strong retrieval performance on the Market-1501 benchmark, with re-ranking further improving the final ranking quality of the retrieved identities.

Inference Optimization and Precision Analysis (Edge Optimization)

To evaluate inference efficiency, I ran the trained ResNet50 ReID model in both FP32 and FP16 precision. In addition to standard inference, I also applied k-reciprocal re-ranking to refine the retrieval results. During this analysis, I measured feature extraction time, total pipeline time, GPU memory usage, and model size.

For the FP16 experiment, the model was converted to half precision to improve computational efficiency. However, the BatchNorm layers were kept in FP32 precision to preserve the accuracy . All other layers were executed in FP16. This mixed-precision approach allowed faster inference and reduced GPU memory consumption without affecting retrieval accuracy.

The results show that FP16 significantly improves feature extraction speed and reduces memory usage while maintaining identical retrieval performance. The comparison between FP32 and FP16 inference is summarized in the table below.

Table 1: Inference Performance Comparison (FP32 vs FP16)

Metric	FP32	FP16	Improvement
mAP (%)	94.08	94.08	0
Rank-1 (%)	95.46	95.46	0
Rank-5 (%)	97.80	97.83	+0.03
Rank-10 (%)	98.34	98.34	0
Feature Extraction Time	43.20 s	21.89 s	1.97× faster
Re-ranking Time	51.71 s	51.33 s	∼same
Total Pipeline Time	94.90 s	73.22 s	21.68 s faster
Peak GPU Memory	1.815 GB	0.973 GB	-0.842 GB

Model Profiling and Inference Analysis

To better understand the deployment characteristics of the trained model, I analyzed the model size, parameter count, embedding storage requirements, and inference latency. The model contains approximately 25M parameters, which is typical for a ResNet50 backbone.

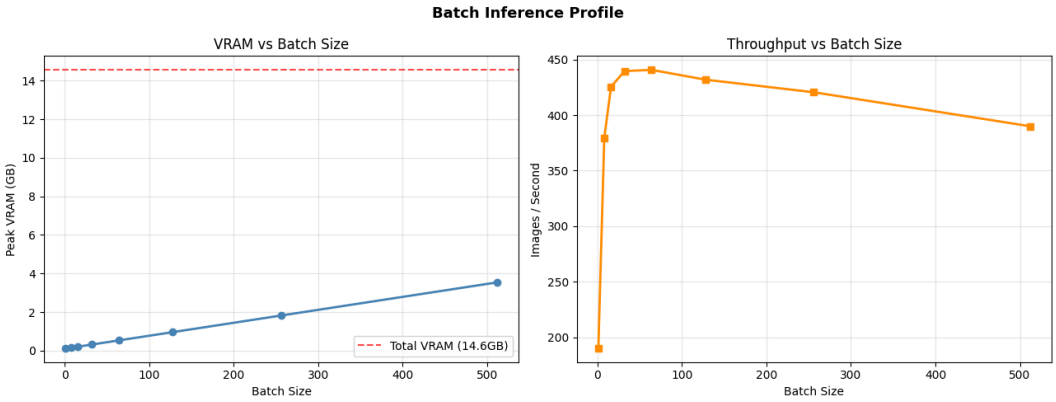
I also measured the memory footprint of embeddings and the inference latency on a Tesla T4 GPU. The results show that a single image can be processed in approximately 5.47 ms, corresponding to about 182 FPS. Additionally, batch inference experiments were conducted to observe how GPU memory usage and throughput scale with different batch sizes.

Table 2: Model Parameters and Size

Metric	Value
Total Parameters	25,050,176
Trainable Parameters	25,048,128
Non-trainable Parameters	2,048
FP32 Model Size	95.56 MB
FP16 Model Size	47.78 MB
Checkpoint Size (with optimizer)	287.15 MB

Table 3: Embedding Storage Requirements

Metric	Value
Embedding Dimension	2048
Size per Embedding (FP32)	8192 bytes
Size per Embedding (FP16)	4096 bytes
Gallery Embeddings (19732 images, FP32)	154.16 MB
Gallery Embeddings (19732 images, FP16)	77.08 MB



The graph on the right shows that the best throughput was achieved at batch sizes 16 and 32, reaching around 425 - 440 images/sec while using only 0.205-0.313 GB VRAM. Increasing the batch size further resulted in higher VRAM consumption .

Real-World Identification Simulation

To evaluate practical deployment feasibility, I simulated a real-world crowd identification scenario using the Market-1501 gallery as a proxy for a deployed camera network.

Setup:
All 15,913 gallery images were embedded once and saved to disk, resulting in a one-time gallery build cost of 68.6 seconds. This represents a pre-built database of known individuals. In a real deployment, this step would be performed offline before the system is deployed.

Scenario:
A batch of 400 query images was submitted simultaneously to the system, simulating a crowd of 400 people being identified against the gallery database.

Once the gallery embeddings are built, each new query only requires one forward pass through the network followed by a similarity search against the stored embeddings, making the system suitable for near real-time operation.

At 4.52 ms per person, the system can theoretically process more than 200 individuals per second on a single Tesla T4 GPU.

Note:
This simulation uses the Market-1501 dataset as a proxy environment. In a real deployment scenario, domain-specific data collection and additional model adaptation would likely be required.

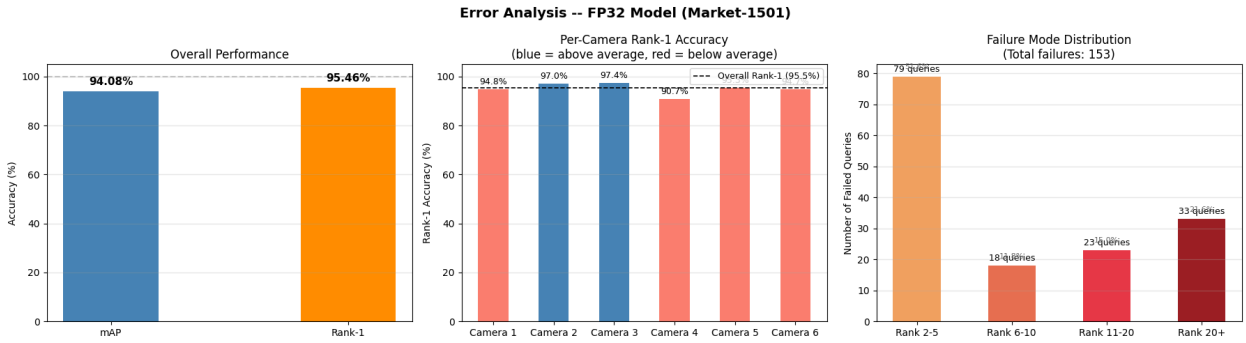
Table 4: Real-World Identification Simulation

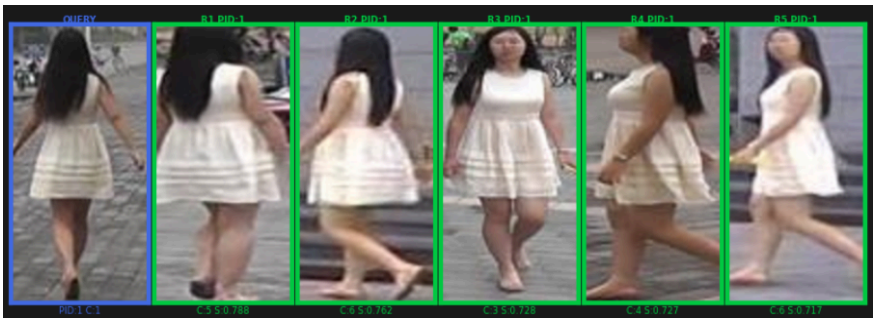
Metric	Value
Gallery Database Size	15,913 images
Gallery Build Time	68.6 s (one-time, offline)
Queries Processed	400 people
Embedding Time	1674.8 ms
Matching Time	131.7 ms
End-to-End Latency per Person	4.52 ms
Top-1 Accuracy	99.5% (398/400)
Top-5 Accuracy	100.0% (400/400)

Error Analysis

I performed an error analysis on the Market-1501 evaluation set to better understand the failure cases of the model. Out of 3368 query images, the model correctly retrieved the matching identity at Rank-1 for 3215 queries (95.5%), while 153 queries (4.5%) failed at Rank-1.

Among these failed cases, many still retrieved the correct identity within the top-5 or top-10 results, indicating that the model often ranks the correct person close to the top but not always in the first position. Only a smaller portion of failures occur beyond the top-20 ranks, which usually correspond to challenging scenarios such as large pose changes, occlusions, background clutter, or cross-camera appearance variations.





The primary failure modes observed are clothing-based confusion (similar colored outfits across different identities), occlusion by objects like bicycles and motorcycles, and extreme viewpoint changes between query and gallery. Camera 4 shows a consistent accuracy drop of ~5% below average, suggesting camera-specific appearance shifts that a global model cannot handle. Most failures are near-misses at Rank 2-5, indicating the model is close to the correct answer but loses to a visually similar distractor. Future improvements include part-based feature extraction

to handle occlusion, camera-aware embeddings (already explored in the ViT pipeline), multi-dataset training for better generalization.

Cross-Dataset Generalization (Zero-Shot Transfer on DukeMTMC-reID)

Code - link

To test whether the model has learned generalizable person features and not just memorized Market-1501 specific patterns I evaluated it on DukeMTMC-reID without any fine-tuning. The model was trained entirely on Market-1501 and tested directly on Duke, which has different cameras, backgrounds, and identities.

Evaluation setup:

Gallery: 17,661 images across 1,110 unseen identities

Query: 2,228 images

No fine-tuning, no domain adaptation pure zero-shot transfer

Results across training checkpoints on Market-1501(zero-shot no training on DukeMTMC-reID):

Table 5: Results Across Training Checkpoints

Checkpoint	Rank-1	Rank-5	Rank-10	mAP	Center Rank-1
Epoch 10	18.81%	32.09%	38.24%	9.10%	61.58%
Epoch 20	19.57%	31.78%	37.43%	9.16%	62.75%
Epoch 30	14.05%	24.46%	29.40%	6.31%	51.97%
Epoch 40	16.56%	26.44%	31.46%	7.57%	59.87%
Epoch 50	20.06%	32.18%	38.02%	10.18%	67.77%
Epoch 60	23.03%	35.32%	41.29%	11.60%	68.49%
Epoch 70	23.56%	36.71%	43.13%	12.51%	69.75%
Epoch 80	24.19%	37.39%	43.40%	12.50%	69.75%
Epoch 90	25.00%	37.57%	44.12%	12.99%	69.97%
Epoch 100	24.91%	38.24%	44.12%	12.82%	69.61%
Epoch 110	26.12%	39.14%	45.42%	13.55%	70.65%
Epoch 120	26.17%	39.27%	45.15%	13.69%	71.10%

These results tell us:

1. The model is genuinely learning not memorizing Standard Rank-1 improves from 18.81% at epoch 10 to 26.17% at epoch 120 on a completely unseen dataset. This means the model is

learning real appearance features clothing color, body shape, walking pattern that transfer across datasets and cameras.

2. Epoch 30 temporary drop There is a visible dip at epoch 30 (Rank-1 drops to 14.05%). This happens right before the first learning rate decay at epoch 40 the model is in a difficult optimization phase and temporarily overfits to Market-1501 specific patterns. It fully recovers by epoch 50 and improves steadily after that.

3. Center Embedding
Standard retrieval at epoch 120 gives **26.17% Rank-1.** Center embedding gives 71.10% Rank-1 a gap of +44.93%.

What is center embedding ?

The center embedding is included to demonstrate embedding space quality. It is not a standard benchmark for cross domain reid.

Instead of matching each query against all 17,661 individual gallery images, compute one representative embedding per person by averaging all their gallery images into a single vector. This single vector captures the person's average appearance filtering out noise from individual bad photos (blurry, occluded, bad lighting). The query is then matched against only 1,110 center vectors instead of 17,661 images.

The +44.93% jump shows that the embedding space itself generalizes well the model places the same person's images close together in feature space even on an unseen dataset. The limitation is not the model's understanding, but the noise in individual gallery images.

Limitations

The primary limitations of the current ResNet50 pipeline are clothing-based confusion, occlusion by objects, and camera-specific appearance shifts. Camera 4 consistently underperforms by ~5% below average. The re-ranking step adds 51s CPU overhead which is a bottleneck for real-time deployment.

Future improvements:

- Part-based feature extraction to handle occlusion
- GPU-accelerated FAISS re-ranking to replace CPU bottleneck
- Multi-dataset training for better generalization
- Longer ViT training to close the gap with ResNet

References

[1] Bag of Tricks and A Strong Baseline for Deep Person Re-Identification Hao Luo, Youzhi Gu, Xingyu Liao, Shenqi Lai, Wei Jiang CVPR Workshops, 2019 arXiv: 1903.07071

[2] Random Erasing Data Augmentation Zhun Zhong, Liang Zheng, Guoliang Kang, Shaozi Li, Yi Yang AAAI, 2020 arXiv: 1708.04896 Used for: Data augmentation random erasing with p=0.5, area ratio 2-40%

[3] Re-ranking Person Re-identification with k-reciprocal Encoding Zhun Zhong, Liang Zheng, Donglin Cao, Shaozi Li CVPR, 2017 arXiv: 1701.08398 Used for: Post-processing — k-reciprocal re-ranking with k1=20, k2=6, lambda=0.3. mAP improved from 85.70% → 94.08%

[4] A Discriminative Feature Learning Approach for Deep Face Recognition (Center Loss) Yandong Wen, Kaipeng Zhang, Zhifei Li, Yu Qiao ECCV, 2016 Used for: Center loss in training β=0.0005, encourages intra-class compactness by pulling same identity embeddings closer

[5] Scalable Person Re-identification: A Benchmark (Market-1501) Liang Zheng, Liyue Shen, Lu Tian, Shuai Wang, Jingdong Wang, Qi Tian ICCV, 2015 Used for: Primary training and evaluation dataset 32,668 images, 1,501 identities, 6 cameras

[6] Performance Measures and a Data Set for Multi-Target, Multi-Camera Tracking (DukeMTMC) Ergys Ristani, Francesco Solera, Roger Zou, Rita Cucchiara, Carlo Tomasi ECCV Workshops, 2016 Used for: Cross-dataset zero-shot evaluation 17,661 gallery images, 1,110 identities, 8 cameras

[7] An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale (ViT) Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov et al. ICLR, 2021 arXiv: 1706.03762 Used for: ViT pipeline backbone DeiT-Base uses this architecture, patch size 16×16, 128 patches, CLS token

[8] Rethinking the Inception Architecture (Label Smoothing) Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, Zbigniew Wojna CVPR, 2016 Used for: Label smoothing ε=0.1 in ID classification loss reduces overconfidence during training